

DevOps Command Reference

20 PAGES OF ESSENTIAL COMMANDS

LINUX | GIT | DOCKER | KUBERNETES | AWS | TERRAFORM | AND MORE



LINUX



GIT



DOCKER



KUBERNETES



AWS



TERRAFORM



NGINX



REDIS



MONITORING



DEVOPS TOOLS



REAL
COMMANDS



PRACTICAL
EXAMPLES



PRODUCTION
SCENARIOS



SENIOR
ENGINEER INSIGHTS

COMMANDS | BUILD | OBSERVE | TROUBLESHOOT | DEPLOY | SCALE

BETTER
ENGINEERS
- BUILD
A BRIGHTER
- TOMORROW

Linux System Triage and Host Health

Build the first-response command set for an unhealthy Linux host.

KEY COMMANDS

\$ uptime	Show uptime, load average and running tasks
\$ hostnamectl	Show hostname and system info
\$ uname -a	Show kernel version and OS details
\$ cat /etc/os-release	Show Linux distribution info
\$ who	Show currently logged-in users
\$ w	Show users and what they are doing
\$ last	Show recent login history
\$ date	Show current date and time
\$ timedatectl	Show and manage time settings
\$ dmesg	Show kernel messages (boot issues, hardware errors)
\$ journalctl	View system and service logs
\$ systemctl --failed	Show failed services
\$ systemd-analyze	Analyze boot performance
\$ ls / proc	Explore process and system info via /proc filesystem

FIRST-RESPONSE CHECKLIST

- ✓ Check overall system health (uptime, load, failures)
- ✓ Identify recent changes (logins, reboots, updates)
- ✓ Check if the issue is system-wide or service-specific
- ✓ Review kernel messages and system logs
- ✓ Check failed services
- ✓ Validate time is correct
- ✓ Collect evidence before making changes

SENIOR ENGINEER FOCUS

- 1 Establishing host state before changing anything
- 2 Load average interpretation
- 3 Kernel versus service-level failures
- 4 Boot problems
- 5 Recent-change correlation
- 6 Defining blast radius
- 7 Evidence preservation during incidents

LOAD AVERAGE GUIDE

- 1 min → Recent load (last 1 minute)
- 5 min → Short-term trend
- 15 min → Longer-term trend

Compare with the number of CPU cores.
Load > CPU cores = potential saturation.

PRODUCTION SCENARIO

— •
• • !
“Server latency suddenly increased. What do you inspect first?”

IMPORTANT NOTES

- 🚫 Do not reboot without evidence.
- 🔗 Correlate issues with recent changes.
- 🔗 Check both system and application logs.
- 🔗 Preserve evidence during incidents.
- 🔗 Load average, CPU, memory and disk issues can all cause latency.

COMMON NEXT STEPS (AFTER INITIAL TRIAGE)

→ Check CPU, Memory, Disk, Network → Investigate specific services → Analyze application logs



BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

CPU and Process Investigation

Diagnose CPU saturation, runaway processes, and process-level failures.

KEY COMMANDS

\$ ps	Show running processes
\$ top	Real-time process and CPU view
\$ pidstat	Per-process CPU statistics
\$ mpstat	CPU usage per core
\$ pgrep	Find process IDs by name
\$ pstree	Show process tree
\$ nice	Start a process with lower priority
\$ renice	Change priority of a running process
\$ taskset	Set CPU affinity for a process
\$ lsof	List open files (find what a process uses)
\$ strace	Trace system calls of a process
\$ kill	Send signal to a process
\$ killall	Kill processes by name
\$ pkill	Kill processes by pattern

COMMON PROCESS STATES (ps)

R	Running or ready to run
S	Sleeping (waiting for an event)
D	Uninterruptible sleep (usually I/O)
T	Stopped or traced
Z	Zombie (process is dead but not reaped)

IMPORTANT PROCESS FIELDS

PID	Process ID (unique identifier)
PPID	Parent process ID
%CPU	CPU utilization by the process
STATE	Process state (R, S, D, T, Z, etc.)
THREADS	Number of threads
ELAPSED	How long the process has been running
COMMAND	The command or application name
PRIORITY	Scheduling priority
NICE	Nice value (lower = higher priority)

SENIOR ENGINEER FOCUS

- 1 CPU saturation versus high load
- 2 User versus system CPU
- 3 Context switching
- 4 Zombie processes
- 5 Stuck processes
- 6 CPU affinity
- 7 Signal escalation
- 8 Why kill -9 should not be the first response

FAILURE INVESTIGATION FLOW

top → pidstat → ps → /proc/PID → strace → hypothesis → mitigation

PRODUCTION TIP



Always investigate and understand the cause before killing a process. Use kill -9 only as a last resort.

COMMON MISTAKES



- Killing processes without understanding the impact
- Ignoring system-wide CPU load
- Not checking for runaway child processes
- Misinterpreting high CPU as a leak



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

Memory Pressure and OOM Investigation

Investigate memory exhaustion without blindly restarting services.

KEY COMMANDS

<code>\$ free</code>	Show memory usage (RAM, cache, swap)
<code>\$ vmstat</code>	View memory, CPU, IO and system activity
<code>\$ ps</code>	Show process memory usage
<code>\$ top</code>	Real-time process and memory view
<code>\$ pmap <PID></code>	Show memory map of a process
<code>\$ smem</code>	Detailed memory usage per process
<code>\$ cat /proc/meminfo</code>	Detailed memory information
<code>\$ cat /proc/<PID>/status</code>	Process memory details (VmRSS, VmSize)
<code>\$ cat /proc/<PID>/smaps</code>	Detailed memory map with per-region info
<code>\$ dmesg grep -i oom</code>	Check OOM killer messages
<code>\$ journalctl grep -i oom</code>	View OOM events in logs

MEMORY ANALYSIS EXAMPLES

1. CHECK MEMORY USAGE

```
$ free -h
              total        used        free
Mem:   15G       12G       2.1G
Swap:  4.0G       1.2G       2.8G
```

Look at available memory and swap usage.

2. CHECK PROCESS MEMORY

```
$ ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem
PID  PPID  %MEM  %CPU  CMD
1234   1   12.5   3.1   java
5678   1    8.2   0.5   nginx
```

Find top memory consumers.

3. CHECK OOM KILLER LOGS

```
$ dmesg | grep -i oom
Out of memory: Killed process
1234 (java) total-vms:8G,
anon-rss:5G, file-rss:0B,
shmem-rss:0B
```

Identify which process was killed and why.

LINUX MEMORY ARCHITECTURE (SIMPLIFIED)

Application Memory (RSS)	→ Used by processes
Page Cache (reclaimable)	→ Speeds up file access
Buffers	→ Metadata and small cache
Free Memory	→ Available for new workloads
Swap (disk)	→ Used when RAM is full

High memory usage does not always mean a problem. Page cache can be reclaimed when needed.

KEY TAKEAWAYS

- ✓ Used memory is not the same as memory pressure.
- ✓ Page cache can be reclaimed by the kernel.
- ✓ Always check OOM logs before restarting services.
- ✓ Monitor memory trends to catch leaks early.
- ✓ Understand container memory limits and cgroups.

MEMORY METRICS TO INVESTIGATE

RSS	Actual physical memory used by process
VSZ	Total virtual memory size
Available memory	Real memory available for new workloads
Page cache	Cache used for filesystem (can be reclaimed)
Swap	Amount of swap used and behavior
Page faults	Major and minor page faults (memory pressure)
Memory growth	Increasing memory usage over time (leak)
OOM killer activity	Processes killed due to out-of-memory

SENIOR ENGINEER FOCUS

- 1 Used memory versus actual memory pressure
- 2 Linux page cache (it can be reclaimed)
- 3 Swap behavior and when it becomes a problem
- 4 Memory leaks and how to identify them
- 5 How the OOM killer selects a process
- 6 Container memory limits and cgroup behavior
- 7 Host versus workload memory exhaustion

PRODUCTION SCENARIO

“Application was killed, but the host still shows available memory.”

TROUBLESHOOTING STEPS

- 1 Check system memory (free, vmstat)
- 2 Identify top memory-consuming processes (ps, top, smem)
- 3 Check OOM killer logs (dmesg, journalctl)
- 4 Validate container limits (if in containers)
- 5 Look for memory leaks (compare over time)
- 6 Take action (tune, restart, increase limits, fix leak)

COMMON MISTAKES

- ✗ Assuming high memory usage always means a problem.
- ✗ Ignoring the OOM killer logs.
- ✗ Restarting services without understanding the cause.
- ✗ Not checking container memory limits.
- ✗ Overlooking memory leaks and long-term trends.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

Disk, Filesystem and I/O Troubleshooting

Diagnose full disks, inode exhaustion, latency, and storage saturation.

KEY COMMANDS

<code>\$ df</code>	Show filesystem disk space usage
<code>\$ du</code>	Show directory and file sizes
<code>\$ lsblk</code>	List block devices and partitions
<code>\$ blkid</code>	Show filesystem UUID and type
<code>\$ findmnt</code>	Show mounted filesystems
<code>\$ mount</code>	Show mount information
<code>\$ iostat</code>	Show I/O statistics
<code>\$ iotop</code>	Show real-time disk I/O usage
<code>\$ stat</code>	Show file or filesystem status
<code>\$ find</code>	Find files (e.g. large files, by type)
<code>\$ lsof</code>	List open files (find deleted files)
<code>\$ fuser</code>	Identify processes using a file/FS
<code>\$ sync</code>	Flush filesystem buffers to disk

COMMON CHECKS WITH EXAMPLES

1. CHECK DISK USAGE

```
$ df -h
Filesystem      Size  Used Avail Use%
/dev/sda1      100G   85G   15G   86%
```

Identify filesystems that are full or nearly full.

2. CHECK INODE USAGE

```
$ df -i
Filesystem      Inodes IUsed IFree IUse%
/dev/sda1      6553600 6500000 53600 99%
```

High inode usage prevents new files from being created.

3. CHECK I/O STATISTICS

```
$ iostat -xz 1 3
Device r/s  w/s  r_wait  w_wait  %util
sda    12   45    1.2     5.6    78.4
```

Shows IOPS, latency and utilization.

4. FIND LARGE FILES

```
$ find / -type f -size +1G 2>/dev/null
/var/log/app.log
/var/lib/docker/overlay2/.../diff/
```

Find files consuming disk space.

5. FIND DELETED-BUT-OPEN FILES

```
$ lsof | grep deleted
java 1234 user 4u REG deleted
```

Files deleted but still open can still consume disk space.

6. CHECK MOUNT DETAILS

```
$ findmnt -T /var
TARGET SOURCE FSTYPE OPTIONS
/var    /dev/sda1 ext4    rw,relatime
```

Verify correct mount, options and type.

KEY TAKEAWAYS

- ✓ Monitor both disk space and inode usage.
- ✓ Use the right tool for the right question.
- ✓ Investigate I/O performance, not just space usage.
- ✓ Deleted files can still consume space if held open.
- ✓ Filesystem issues can cause application failures even with free space.

WHAT TO INVESTIGATE

- 1 Capacity – disk usage and available space
- 2 Inode exhaustion – no space for new files
- 3 IOPS – input/output operations per second
- 4 Throughput – read/write speed (MB/s)
- 5 Latency – time per I/O operation
- 6 Queue depth – pending I/O requests
- 7 I/O wait – time spent waiting for disk
- 8 Deleted-but-open files – disk space not freed
- 9 Mount failures – incorrect or failed mounts
- 10 Filesystem state – read-only, errors, corruption

SENIOR INSIGHT



df and du disagreeing can itself be diagnostic evidence.

This often happens due to deleted-but-open files, bind mounts, snapshots, or different filesystem views (e.g. container vs host).

PRODUCTION SCENARIO



“Disk usage is at 100% and applications are failing. How do you investigate and resolve this safely?”

TROUBLESHOOTING STEPS

- 1 Check overall disk and inode usage (`df`, `df -i`)
- 2 Identify large files/directories (`du`, `find`)
- 3 Check for deleted-but-open files (`lsof`)
- 4 Review I/O performance (`iostat`, `iotop`)
- 5 Verify mounts and filesystem state (`findmnt`, `mount`)
- 6 Check system logs for disk or filesystem errors
- 7 Take appropriate action (cleanup, rotate logs, expand)
- 8 Verify and monitor after the fix

COMMON MISTAKES

- ✗ Ignoring inode exhaustion.
- ✗ Deleting files still in use (without verifying).
- ✗ Not checking for deleted-but-open files.
- ✗ Assuming low CPU means disk is healthy.
- ✗ Not investigating I/O latency or queue depth.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

Linux Networking and Socket Diagnostics

Diagnose connectivity from the host outward.

KEY COMMANDS

<code>\$ ip addr</code>	Show network interfaces and IP addresses
<code>\$ ip link</code>	Show and manage network interfaces
<code>\$ ip route</code>	Show routing table
<code>\$ ip neigh</code>	Show ARP/neighbor table
<code>\$ ss</code>	Show socket connections and listening ports
<code>\$ ping</code>	Test basic connectivity
<code>\$ traceroute</code>	Trace network path (hop by hop)
<code>\$ tracepath</code>	Trace path and discover MTU
<code>\$ mtr</code>	Real-time traceroute with statistics
<code>\$ arp</code>	Show and manage ARP cache
<code>\$ ethtool</code>	Check interface details and errors
<code>\$ nc</code>	Test connections to a port
<code>\$ nmap</code>	Scan ports and check service availability
<code>\$ tcpdump</code>	Capture and analyze network packets

COMMON EXAMPLES

1. CHECK INTERFACES

```
$ ip addr show
```

Verify IP address, interface state and errors.

2. CHECK ROUTES

```
$ ip route show
```

Confirm correct routing and default gateway.

3. TEST CONNECTIVITY

```
$ ping -c 4 8.8.8.8
```

Check basic network reachability.

4. TRACE PATH

```
$ mtr google.com
```

Identify latency and packet loss along the path.

5. CHECK LISTENING PORTS

```
$ ss -tulnp
```

View listening ports and services.

6. CAPTURE PACKETS

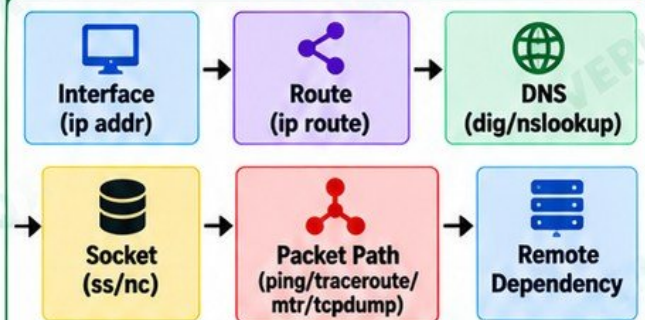
```
$ tcpdump -i eth0 -n
```

Capture real-time packets (for deeper analysis).

TROUBLESHOOTING CHECKLIST

- ✓ Verify interface is up and has the correct IP
- ✓ Check routing table and default gateway
- ✓ Test DNS resolution (e.g. dig, nslookup)
- ✓ Check if the port is open (ss, nc)
- ✓ Trace the path (mtr, traceroute)
- ✓ Capture packets if needed (tcpdump)
- ✓ Check for packet loss, latency or MTU issues
- ✓ Compare with a working host

INVESTIGATION MODEL



Follow this order to quickly isolate where the failure is.

SENIOR ENGINEER FOCUS

- 1 LISTEN versus ESTABLISHED (what is normal)
- 2 SYN states and TCP connection flow
- 3 Connection exhaustion (too many open connections)
- 4 Packet loss and high latency
- 5 Routing and incorrect routes
- 6 Interface errors and drops
- 7 MTU issues and fragmentation
- 8 Ephemeral ports and port exhaustion
- 9 Retransmissions and what they indicate

KEY SOCKET STATES (TCP)

LISTEN	Service is waiting for connections
ESTABLISHED	Connection is active
SYN-SENT	Client sent SYN, waiting for response
SYN-RECEIVED	Server received SYN, waiting for ACK
TIME-WAIT	Connection closed, waiting to clean up
CLOSE-WAIT	Remote closed, waiting for local close

PRODUCTION SCENARIO



"Users report the application is unreachable from the server. How do you investigate?"

KEY TAKEAWAY



Network issues are rarely fixed by one command. Follow the investigation model, collect evidence at each layer, and isolate the failure domain.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

DNS, HTTP, TLS and Application Connectivity

Troubleshoot the entire application request path.

KEY COMMANDS

\$ dig	Detailed DNS queries
\$ host	Simple DNS lookup
\$ nslookup	DNS lookup and troubleshooting
\$ resolvectl	Check DNS resolver configuration
\$ curl	Test HTTP/HTTPS requests
\$ wget	Download and test HTTP/HTTPS
\$ openssl s_client	Test TLS/SSL connections
\$ nc	Test TCP/UDP connectivity
\$ ss	View socket connections and states

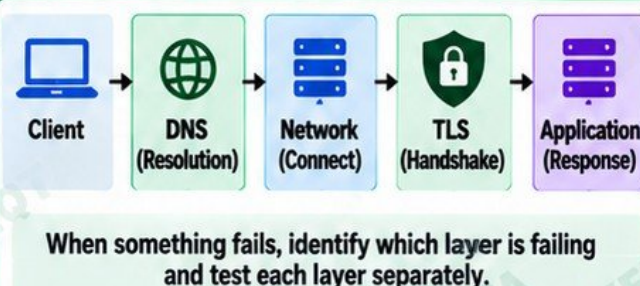
DIAGNOSTIC COMMAND EXAMPLES

\$ dig service.example.com	Check DNS resolution (A/AAAA/CNAME)
\$ host service.example.com	Quick DNS lookup
\$ nslookup service.example.com	Alternative DNS query
\$ resolvectl status	Check local DNS configuration
\$ curl -v https://service.example.com	Test HTTP/HTTPS with detailed output
\$ curl -I https://service.example.com	Check HTTP headers only
\$ curl --resolve service.example.com:443:1.2.3.4	Test a specific IP (bypass DNS)
\$ wget https://service.example.com	Download and test connectivity
\$ openssl s_client -connect host:443 \ -servername host	Test TLS handshake and certificate
\$ nc -zv host 443	Test if a TCP port is open
\$ ss -tulnp	View listening ports and connections

TROUBLESHOOTING CHECKLIST

- ✓ Start with DNS (dig, host, nslookup)
- ✓ Verify the IP can be reached (ping, nc, traceroute)
- ✓ Check the service port (nc, ss)
- ✓ Validate TLS handshake (openssl s_client)
- ✓ Inspect HTTP response (curl -v, curl -I)
- ✓ Check for redirects and proxy behavior
- ✓ Compare with a working endpoint
- ✓ Identify the failure layer and escalate with evidence

APPLICATION REQUEST PATH



SENIOR ENGINEER FOCUS

- 1 DNS resolution (A, AAAA, CNAME, TTL)
- 2 HTTP status codes and response behavior
- 3 TLS negotiation and cipher selection
- 4 Certificates (validity, chain, hostname)
- 5 Redirects and HTTP to HTTPS behavior
- 6 Proxy behavior (forward and reverse proxies)
- 7 Timeout boundaries (DNS, connect, TLS, read)
- 8 Connection versus application failures

COMMON FAILURE SYMPTOMS

Symptom	Likely Layer
Cannot resolve hostname	DNS
Connection timed out	Network / Firewall
TLS handshake fails	TLS / Certificate
HTTP 4xx/5xx	Application
Very slow response	Network / Application
Works with curl but not app	Application / Proxy
Intermittent failures	DNS / Network / Load Balancer

KEY TAKEAWAY

 Test each layer separately. A successful DNS lookup does not mean the application is working. Collect evidence at every step to find the real failure.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

Logs, Journald and Production Evidence

Extract useful evidence from large production logs.

KEY COMMANDS

\$ journalctl	Query systemd logs (journald)
\$ dmesg	View kernel ring buffer messages
\$ tail	View the last lines of a file
\$ head	View the first lines of a file
\$ less	View large files (page by page)
\$ grep	Search for a pattern in files
\$ rg	Fast search using ripgrep
\$ awk	Text processing and filtering
\$ sed	Stream editor for text manipulation
\$ cut	Extract specific fields/columns
\$ sort	Sort lines of text
\$ uniq	Find and count unique lines
\$ wc	Count lines, words, or bytes
\$ tee	Output to screen and file
\$ xargs	Build and execute commands from input

COMMON EXAMPLES

\$ journalctl -u nginx -n 100	View last 100 logs for a service
\$ journalctl --since "1 hour ago"	Logs from the last 1 hour
\$ dmesg tail -n 50	View recent kernel messages
\$ tail -f /var/log/syslog	Follow logs in real time
\$ grep -i error app.log	Search for errors (case insensitive)
\$ rg "timeout" /var/log	Fast search for timeout keyword
\$ awk '{print \$1, \$NF}' file.log	Print first and last field
\$ sed 's/ERROB/WARN/g' file.log	Replace text in log output
\$ cut -d ' ' -f1 access.log	Extract first column (e.g. IP)
\$ sort access.log uniq -c	Count unique occurrences
\$ wc -l app.log	Count total lines in a file
\$ tee output.log	Save output while viewing
\$ cat file.txt xargs -n1 ping	Run a command for each line

USEFUL LOG PATTERNS

	Time-window filtering	journalctl --since "2024-01-01" --until "2024-01-02"
	Error-frequency analysis	grep -i error app.log uniq -c sort -nr
	Correlation IDs	grep "request_id" app.log
	IP aggregation	awk '{print \$1}' access.log sort uniq -c sort -nr
	Status-code analysis	awk '{print \$9}' access.log sort uniq -c sort -nr
	Service-specific filtering	journalctl -u service-name grep -i error

SENIOR ENGINEER FOCUS

- 1 Turn millions of log lines into a testable hypothesis.
- 2 Understand log sources (application, system, kernel, container, audit).
- 3 Filter by time, service, and severity.
- 4 Look for patterns, not just single errors.
- 5 Correlate logs with metrics and events.
- 6 Identify the real root cause, not just symptoms.
- 7 Know when to increase log levels safely.
- 8 Handle large logs efficiently (use tools like rg, less, awk).

REAL-WORLD SCENARIO



"Users report intermittent errors. Logs are large and noisy. How do you find the real cause quickly?"

KEY TAKEAWAYS

- ✓ Use the right tool for the right job.
- ✓ Filter early (time, service, severity).
- ✓ Look for patterns and trends.
- ✓ Correlate logs with other signals.
- ✓ Turn log data into a clear, testable hypothesis.

COMMON MISTAKES

- ✗ Searching without a time range.
- ✗ Ignoring context and related logs.
- ✗ Looking at only one log line.
- ✗ Not checking multiple log sources.
- ✗ Using slow tools on very large files.
- ✗ Assuming the first error is the root cause.

PRO TIP



Combine commands to get powerful results.
For example: journalctl -u app --since "1 hour ago" | rg -i error | awk '{print \$1}' | sort | uniq -c | sort -nr



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

Files, Permissions and Security Investigation

Diagnose access problems safely.

KEY COMMANDS

\$ ls -la	List files with detailed permissions
\$ stat	Show detailed file metadata
\$ chmod	Change file permissions
\$ chown	Change file owner
\$ chgrp	Change file group
\$ umask	Show or set default permissions
\$ getfacl	View access control lists (ACLs)
\$ setfacl	Set access control lists (ACLs)
\$ id	Show user ID and group ID
\$ groups	Show user groups
\$ getent	Query system user/group database
\$ sudo	Run command as another user
\$ namei	Show path and permission details
\$ getcap	Show file capabilities
\$ setcap	Set file capabilities

COMMON EXAMPLES

\$ ls -la /etc	List files and permissions
\$ stat /var/log/app.log	Show detailed file information
\$ chmod 640 file.txt	Change file permissions
\$ chown appuser:app /opt/app	Change file owner and group
\$ chgrp developers /opt/app	Change file group
\$ umask	Show current umask
\$ getfacl /data	View ACLs on a directory
\$ setfacl -m u:user:rw file.txt	Add ACL for a user
\$ id appuser	Show UID, GID and groups
\$ groups appuser	Show user's groups
\$ getent passwd appuser	Check user in system database
\$ sudo -u appuser command	Run as another user
\$ namei -l /opt/app/config	Show path permissions
\$ getcap /usr/bin/ping	Check file capabilities
\$ setcap cap_net_raw+ep /usr/bin/ping	Set file capabilities

PERMISSION TYPES

Type	Symbol	Meaning	Example
Read	r	Can read file/directory	r-- (4)
Write	w	Can modify file/directory	-w- (2)
Execute	x	Can run file or enter directory	--x (1)
Special	s	Set user/group ID	s (on file/dir)
	S	Set ID without execute	S
	t	Sticky bit (directories)	t (e.g. /tmp)

SENIOR ENGINEER FOCUS

- 1 UID/GID mismatches and how to fix them
- 2 Understanding and using ACLs
- 3 Inherited permissions and directory structures
- 4 Service accounts and least privilege
- 5 Linux capabilities for secure access
- 6 Container permission problems
- 7 Mounted-volume ownership (host vs container)
- 8 Auditing and verifying access safely

REAL-WORLD SCENARIO



"Application cannot read the configuration file. How do you investigate and fix this safely?"

KEY TAKEAWAY

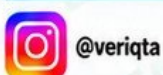
Understand the current permissions, the required access, and the safest way to make the change. Avoid treating **chmod 777** as troubleshooting.

COMMON MISTAKES

- ✗ Using chmod 777 as a quick fix.
- ✗ Ignoring ACLs and inherited permissions.
- ✗ Changing ownership without understanding the impact.
- ✗ Running everything as root.
- ✗ Forgetting container filesystem permissions.
- ✗ Not verifying the full path and mount options.

PRO TIP

Use **namei -l** to check permissions on every directory in the path, not just the file itself.



OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

systemd and Service Failure Investigation

Diagnose services that fail, restart repeatedly, or behave differently after boot.

KEY COMMANDS

\$ systemctl status	Show service status and details
\$ systemctl start	Start a service
\$ systemctl stop	Stop a service
\$ systemctl restart	Restart a service
\$ systemctl reload	Reload service configuration
\$ systemctl enable	Enable service at boot
\$ systemctl disable	Disable service at boot
\$ systemctl cat	Show full unit file content
\$ systemctl show	Show service properties
\$ systemctl list-units	List all units (services, timers, etc.)
\$ systemctl list-dependencies	Show unit dependencies
\$ systemctl --failed	List failed services
\$ journalctl -u <service>	View logs for a specific service
\$ systemd-analyze	Analyze boot time and performance

USEFUL EXAMPLES

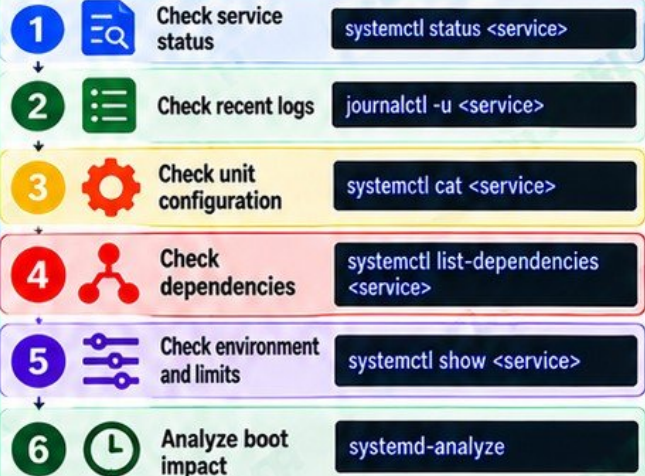
\$ systemctl status nginx	Check service status
\$ journalctl -u nginx -n 100	View last 100 log lines
\$ systemctl restart nginx	Restart the service
\$ systemctl enable nginx	Enable service at boot
\$ systemctl disable nginx	Disable service at boot
\$ systemctl is-active nginx	Check if service is active
\$ systemctl is-enabled nginx	Check if service is enabled
\$ systemctl list-dependencies nginx	Show service dependencies
\$ systemctl show nginx	Show service properties
\$ systemd-analyze blame	Show boot time by service
\$ systemd-analyze critical-chain	Show critical boot path
\$ systemctl --failed	List all failed services

REAL-WORLD SCENARIO



"The application service keeps restarting after boot. Logs show no clear error. How do you find and fix the real cause?"

SERVICE INVESTIGATION FLOW



SENIOR ENGINEER FOCUS

- 1 Unit dependencies and how they affect startup.
- 2 Restart policies (Restart=, StartLimit*).
- 3 Environment variables and configuration.
- 4 Startup ordering (After=, Requires=, Wants=).
- 5 Service users and file permissions.
- 6 Resource limits (CPU, memory, tasks, file descriptors).
- 7 Dependency failures and their impact.
- 8 Boot critical path and service performance.

COMMON FAILURE SYMPTOMS

- ✗ Service fails to start.
- ✗ Service keeps restarting (crash loop).
- ✗ Service is active but not working as expected.
- ✗ Service works manually but not at boot.
- ✗ Dependency service is missing or failed.
- ✗ Permission or environment issues.
- ✗ High resource usage causing service failure.
- ✗ Slow boot due to service delays.

KEY TAKEAWAY



Investigate systematically: status → logs → configuration → dependencies → environment. Understand the service, not just the symptom.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

Git Production and Recovery Commands

Handle real repository and release problems safely.

KEY COMMANDS

\$ git status	Show working directory status
\$ git log	View commit history
\$ git diff	Show changes between commits
\$ git show	Show details of a commit, tag or object
\$ git branch	List, create or delete branches
\$ git switch	Switch to another branch
\$ git restore	Restore files in working directory
\$ git fetch	Download latest changes (no merge)
\$ git pull	Fetch and merge changes
\$ git rebase	Reapply commits on top of another branch
\$ git merge	Merge branches
\$ git cherry-pick	Apply a specific commit
\$ git revert	Create a new commit that undoes changes
\$ git reset	Move HEAD to a previous commit
\$ git reflog	View reference logs (recover lost commits)
\$ git bisect	Find the commit that introduced a bug
\$ git blame	Show who changed a line and when
\$ git tag	Create, list or manage release tags

COMMON EXAMPLES

\$ git status	Check current status
\$ git log --oneline --graph	View commit history (visual)
\$ git diff main..feature	Compare branches
\$ git show <commit-id>	View commit details
\$ git branch -a	List all branches (local and remote)
\$ git switch feature	Switch to a branch
\$ git restore file.txt	Discard changes in a file
\$ git fetch origin	Fetch latest changes
\$ git pull origin main	Pull latest changes
\$ git rebase main	Rebase onto main branch
\$ git merge feature	Merge a feature branch
\$ git cherry-pick <commit-id>	Apply a specific commit
\$ git revert <commit-id>	Revert a commit safely
\$ git reset --hard <commit-id>	Reset to a previous commit
\$ git reflog	Recover lost commits
\$ git bisect start	Find the bad commit (then good/bad)
\$ git blame file.txt	See who changed each line
\$ git tag v1.0.0	Create a release tag

REAL-WORLD SCENARIO



"A deployment introduced a bug in production. You need to quickly identify the bad commit and roll back safely without losing important work. What do you do?"

SENIOR ENGINEER FOCUS

- 1 Revert versus reset (when to use each)
- 2 Recovering lost commits with reflog
- 3 Identifying the commit that introduced a regression
- 4 Using tags for releases and rollbacks
- 5 Risks of history rewriting (rebase, force push)
- 6 Protected branches and production safety
- 7 Safe rollback strategies in production environments
- 8 Maintaining a clean and traceable git history

COMMON MISTAKES

- ✗ Using git reset --hard without understanding the impact
- ✗ Force pushing to protected branches
- ✗ Rebasing shared branches
- ✗ Not using tags for releases
- ✗ Ignoring reflog (makes recovery harder)
- ✗ Cherry-picking without understanding dependencies
- ✗ Mixing up merge, rebase and squash
- ✗ Making changes directly on main in production

KEY TAKEAWAY



Use the right command for the right situation. Prefer revert over reset in production. Always use tags for releases. Understand the impact before rewriting history. Keep your repository clean, safe and recoverable.

PRO TIP



When in doubt, create a backup branch first. You can always recover, but you cannot undo a force push easily.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW



Docker Runtime and Container Diagnostics

Investigate failing containers rather than repeatedly restarting them.

KEY COMMANDS

\$ docker ps	List running containers
\$ docker inspect	View detailed container configuration
\$ docker logs	View container logs
\$ docker exec	Run a command inside a container
\$ docker stats	Show container resource usage
\$ docker top	View running processes in a container
\$ docker events	Monitor real-time Docker events
\$ docker port	View port mappings
\$ docker diff	Show filesystem changes in a container
\$ docker image inspect	Inspect image metadata
\$ docker network inspect	Inspect network configuration
\$ docker volume inspect	Inspect volume details

USEFUL EXAMPLES

\$ docker ps -a	List all containers (including stopped)
\$ docker inspect <container>	View full container details (JSON)
\$ docker logs -f <container>	Follow logs in real time
\$ docker exec -it <container> bash	Open a shell in the container
\$ docker stats	Monitor CPU, memory, network, I/O
\$ docker top <container>	View running processes
\$ docker port <container>	Show port mappings
\$ docker diff <container>	See filesystem changes
\$ docker image inspect <image>	Inspect image configuration
\$ docker network inspect <network>	View network details
\$ docker volume inspect <volume>	View volume details
\$ docker events --since 1h	Show events from the last 1 hour

REAL-WORLD SCENARIO



"A container keeps restarting. Logs show it exits with code 137. How do you find the real cause and fix it without blindly increasing memory limits?"

CONTAINER INVESTIGATION FLOW

1	Check container state	docker ps -a
2	Get exit code and reason	docker inspect <container>
3	Check logs	docker logs <container>
4	Review runtime configuration	docker inspect <container>
5	Analyze resources	docker stats <container>
6	Check networking	docker network inspect <network>
7	Check storage and mounts	docker volume inspect <volume>

SENIOR ENGINEER FOCUS

- 1 Container state and exit codes.
- 2 Understanding OOMKilled and its causes.
- 3 Health checks and restart policies.
- 4 Runtime configuration (env vars, args, entryptoint).
- 5 Resource limits (CPU, memory, pids).
- 6 Networking (networks, DNS, port mappings).
- 7 Storage and mounts (volumes, bind mounts).
- 8 Namespaces and isolation.

COMMON FAILURE SYMPTOMS

- ✗ Container exits immediately (non-zero exit code).
- ✗ OOMKilled due to memory limits.
- ✗ Health check failures (container restarts).
- ✗ Cannot connect to container (network or port issue).
- ✗ Missing or incorrect volume mounts.
- ✗ Application works locally but not in container.

KEY TAKEAWAY

- Investigate before you restart.
- Understand the container's state, logs, configuration and resources to find the root cause.
- Fix the problem, not just the symptom.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW



Docker Images, Builds and Supply Chain

Understand what is actually being shipped.

KEY COMMANDS

\$ docker build	Build an image from a Dockerfile
\$ docker buildx	Build multi-platform images
\$ docker history	View image layer history
\$ docker image ls	List local images
\$ docker image inspect	Inspect image details (JSON)
\$ docker pull	Download an image from a registry
\$ docker push	Upload an image to a registry
\$ docker tag	Tag an image
\$ docker save	Save an image to a file (tar)
\$ docker load	Load an image from a file (tar)
\$ docker system df	Show disk usage (images, containers, etc.)

COMMON EXAMPLES

\$ docker build -t myapp:1.0 .	Build an image
\$ docker buildx build --platform linux/amd64,linux/arm64 -t myapp:1.0 .	Build multi-platform
\$ docker history myapp:1.0	View layer history
\$ docker image ls	List local images
\$ docker image inspect myapp:1.0	Inspect image (JSON)
\$ docker pull nginx:latest	Pull image from registry
\$ docker push myrepo/myapp:1.0	Push image to registry
\$ docker tag myapp:1.0 myrepo/myapp:1.0	Tag image
\$ docker save myapp:1.0 -o myapp.tar	Save image to file
\$ docker load -i myapp.tar	Load image from file
\$ docker system df	Show disk usage

IMAGE LAYERS AND CACHING



- ✓ Each instruction in a Dockerfile creates a layer.
- ✓ Layers are cached to speed up builds.
- ✓ Change only what is necessary to leverage the cache.
- ✓ Use .dockerignore to exclude unnecessary files.
- ✓ Smaller images build faster, push faster and are more secure.

SENIOR ENGINEER FOCUS

- 1 Layer caching and cache invalidation.
- 2 Build context and .dockerignore.
- 3 Multi-stage builds for smaller, secure images.
- 4 Image size optimization.
- 5 Reproducibility (same build, same image).
- 6 Image digests versus tags (immutability).
- 7 Provenance and software supply chain.
- 8 Vulnerability scanning (e.g. Trivy, Gype).
- 9 Registry security (private registries, access control).
- 10 Immutable artifacts in production.

REAL-WORLD SCENARIO



"A new image was deployed and the application started failing. How do you verify what changed, identify the issue and roll back safely?"

COMMON MISTAKES

- ✗ Using latest tag in production.
- ✗ Ignoring .dockerignore (large build context).
- ✗ Building images with unnecessary files.
- ✗ Not scanning images for vulnerabilities.
- ✗ Storing secrets in images.
- ✗ Not using image digests for production.
- ✗ Pushing to public registries by mistake.
- ✗ Skipping multi-stage builds.

KEY TAKEAWAY



Build secure, minimal, reproducible images. Understand what is in your images and where they come from. Treat images as immutable artifacts, not temporary snapshots.

PRO TIP



Always use multi-stage builds, a .dockerignore file and image scanning. Use digests instead of tags in production and keep your supply chain secure and traceable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW



Kubernetes Workload Investigation

Build the core production Kubernetes troubleshooting command tree.

KEY COMMANDS

\$ kubectl get	List resources (pods, deployments, services, etc.)
\$ kubectl describe	Show detailed information about a resource
\$ kubectl logs	View container logs
\$ kubectl exec	Run a command inside a container
\$ kubectl top	Show resource usage (CPU, memory)
\$ kubectl events	View recent cluster events
\$ kubectl explain	Show documentation for a resource
\$ kubectl wait	Wait for a specific condition

COMMON EXAMPLES

\$ kubectl get pods -A	List all pods in all namespaces
\$ kubectl describe pod pod-name	Show pod details and events
\$ kubectl logs pod-name	View logs from a pod
\$ kubectl logs pod-name -c container	View logs from a specific container
\$ kubectl exec -it pod-name -- bash	Open a shell in a pod
\$ kubectl top pods	Show CPU and memory usage
\$ kubectl events --sort-by=.lastTimestamp	View recent events
\$ kubectl explain pod	Show documentation for pod
\$ kubectl wait --for=condition=ready \ pod/pod-name	Wait for pod to be ready

COMMON WORKLOAD ISSUES

	Pending	Pod is waiting for resources or scheduling.
	CrashLoopBackOff	Container keeps crashing and restarting.
	ImagePullBackOff	Failed to pull container image from registry.
	OOMKilled	Container killed due to out-of-memory.
	Failed Probes	Liveness or readiness probe failing.
	Scheduling Failures	Pod cannot be scheduled on any node.
	Resource Pressure	Insufficient CPU, memory, or storage on nodes.
	Configuration Failures	Incorrect config, environment variables or secrets.

REAL-WORLD SCENARIO



"A pod stays in Pending state and never starts. How do you investigate the root cause and get the application running?"

KEY TAKEAWAY



Follow the command tree: get → describe → events → logs → resources → dependencies. Do not guess. Use Kubernetes events and messages to find the real cause.

SENIOR INVESTIGATION PATTERN

1		Get	Start with kubectl get to see the overall state.
2		Describe	Use kubectl describe for detailed information.
3		Events	Check events for errors and reasons.
4		Logs	View logs for application and container errors.
5		Resources	Check resource usage (CPU, memory, limits, requests).
6		Dependencies	Verify dependent resources (ConfigMaps, Secrets, PVCs, Services).

IMPORTANT OBJECTS TO CHECK

	Pods	Status, events, logs, resource usage
	Deployments	Replica status and rollout issues
	ReplicaSets	Pod replica management
	Services	Endpoints and connectivity
	ConfigMaps	Configuration values
	Secrets	Sensitive configuration
	PersistentVolumes (PV)	Storage availability and binding
	PersistentVolumeClaims (PVC)	Claim status and storage class
	Nodes	Node health, resource pressure, taints
	Namespaces	Resource isolation and quotas

PRO TIP



Always check events and describe output first. They often show the exact reason (e.g. insufficient resources, image not found, invalid configuration).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW



Kubernetes Networking and Service Debugging

Trace traffic through Kubernetes.

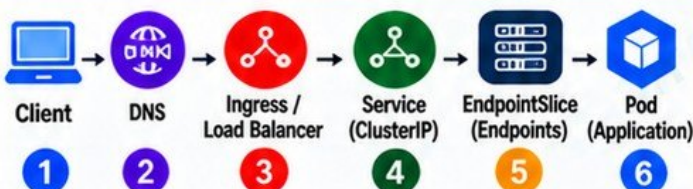
KEY KUBECTL COMMANDS

\$ kubectl get svc	List all services
\$ kubectl get endpoints	View service endpoints
\$ kubectl get endpointslices	View EndpointSlice objects
\$ kubectl get ingress	List ingress resources
\$ kubectl get networkpolicy	List network policies
\$ kubectl port-forward	Forward a local port to a pod/service
\$ kubectl exec	Run a command inside a pod

ADDITIONAL NETWORKING TOOLS

\$ dig	Test DNS resolution
\$ curl	Test HTTP/HTTPS connectivity
\$ ss	Check listening ports and connections
\$ nc	Test TCP/UDP connectivity
\$ tcpdump	Capture and analyze network traffic

TRAFFIC FLOW (END-TO-END)



REAL-WORLD SCENARIO

"The application is running, but users cannot access it. How do you trace the traffic path and find the real cause?"

KEY TAKEAWAY

Follow the traffic step by step from client to pod. Verify DNS, ingress, service, endpoints and pod. Check configuration, network policies and actual traffic, not just the symptoms.

COMMON INVESTIGATION STEPS

1	Check DNS	<code>dig <service>.<namespace>.svc.cluster.local</code>
2	Check Service	<code>kubectl get svc</code> <code>kubectl describe svc <name></code>
3	Check Endpoints	<code>kubectl get endpoints</code> <code>kubectl get endpointslices</code>
4	Check Ingress	<code>kubectl get ingress</code> <code>kubectl describe ingress <name></code>
5	Check NetworkPolicy	<code>kubectl get networkpolicy</code> <code>kubectl describe networkpolicy</code>
6	Test Connectivity	<code>curl</code> , <code>nc</code> , <code>ss</code> , <code>tcpdump</code>
7	Check Pod	<code>kubectl logs <pod></code> <code>kubectl exec -it <pod> -- bash</code>

SENIOR ENGINEER FOCUS

- 1 DNS failures (CoreDNS, external DNS, records)
- 2 Selectors (label mismatch between service and pods)
- 3 Missing endpoints (no ready pods)
- 4 Readiness (readiness probes, pod not ready)
- 5 CNI (network plugin issues, pod-to-pod connectivity)
- 6 NetworkPolicy (blocking traffic)
- 7 Ingress routing (rules, annotations, TLS, host/path issues)
- 8 kube-proxy or alternative data planes (IPVS, eBPF, etc.)

COMMON FAILURE SYMPTOMS

- ✗ Service has no endpoints
- ✗ DNS resolution fails
- ✗ Connection timeout or refused
- ✗ Ingress returns 404 or 503
- ✗ NetworkPolicy blocks traffic
- ✗ Pod is running but not ready
- ✗ Traffic works inside cluster but not from outside
- ✗ Intermittent connectivity

PRO TIP

Always verify each layer in the path. Use real commands to confirm what is actually happening. Network issues are often configuration issues, not code issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW



Kubernetes Nodes, Scheduling and Resource Pressure

Diagnose why workloads cannot run reliably.

KEY KUBECTL COMMANDS

\$ kubectl get nodes	List all nodes in the cluster
\$ kubectl describe node <node>	Show detailed node information
\$ kubectl top node	Show node resource usage (CPU, memory)
\$ kubectl top pod	Show pod resource usage
\$ kubectl get events	View recent events
\$ kubectl cordon <node>	Mark a node as unschedulable
\$ kubectl uncordon <node>	Allow scheduling on a node again
\$ kubectl drain <node>	Safely evict pods for maintenance

INSPECT IN NODE AND POD SPECS

	Requests	Minimum resources required
	Limits	Maximum resources allowed
	Taints	Mark nodes to repel certain pods
	Tolerations	Allow pods to run on tainted nodes
	Affinity	Node, pod and anti-affinity rules
	Topology	Topology spread constraints
	Node Conditions	Ready, DiskPressure, MemoryPressure, PIDPressure
	Resource Usage	CPU, memory, disk, pods, inodes

NODE CONDITIONS

	Ready	Node is healthy and ready to run pods
	DiskPressure	Node is low on disk space
	MemoryPressure	Node is low on memory
	PIDPressure	Node has insufficient PIDs
	NotReady	Node is not ready (check events and kubelet)
	SchedulingDisabled	Node is cordoned (unschedulable)

SCHEDULING FAILURE REASONS

- 1 Insufficient CPU or memory
- 2 Node taints not tolerated
- 3 Node affinity or anti-affinity rules
- 4 Topology spread constraints not satisfied
- 5 Node selectors do not match
- 6 Insufficient ephemeral storage
- 7 Too many pods on node (pod density)
- 8 Node conditions (DiskPressure, MemoryPressure, PIDPressure)

RESOURCE VIEW EXAMPLES

\$ kubectl top nodes	Check node CPU and memory usage
\$ kubectl top pods -A	Check all pod resource usage
\$ kubectl describe node <node>	View node conditions and capacity
\$ kubectl get pods -A -o wide	See pod node placement
\$ kubectl get events --sort-by=.lastTimestamp	View recent scheduling events

SENIOR ENGINEER FOCUS

- ✓ Scheduling failures and how to fix them
- ✓ Eviction and node pressure handling
- ✓ Quality of Service (QoS) classes (Guaranteed, Burstable, BestEffort)
- ✓ Understanding node capacity and allocatable resources
- ✓ Handling node disruption and maintenance safely
- ✓ Planning for high availability and resilient scheduling

REAL-WORLD SCENARIO




"A critical pod stays in Pending state due to insufficient memory. How do you find the cause, confirm the node capacity and fix it without impacting other workloads?"

KEY TAKEAWAY

 Check events, node conditions and resource usage. Understand scheduling rules (requests, limits, taints, tolerations, affinity). Fix the root cause, not just the symptom.

PRO TIP

 Always drain nodes safely before maintenance. Monitor node resource usage and set alerts for DiskPressure, MemoryPressure and PIDPressure.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW



Kubernetes Deployments, Rollouts and Recovery

Operate releases safely.

KEY KUBECTL COMMANDS

\$ kubectl rollout status	Check rollout status
\$ kubectl rollout history	View rollout history
\$ kubectl rollout undo	Rollback to a previous version
\$ kubectl scale	Manually scale a deployment
\$ kubectl autoscale	Enable autoscaling (HPA)
\$ kubectl diff	Preview changes before applying
\$ kubectl apply	Apply configuration changes
\$ kubectl patch	Update specific fields
\$ kubectl set image	Update the container image

DEPLOYMENT ROLLOUT FLOW

1	Update manifest	Change image, config or replicas
2	Apply change	<code>kubectl apply -f deployment.yaml</code>
3	Watch rollout	<code>kubectl rollout status deployment/<name></code>
4	Verify success	Check pods, logs, metrics and user functionality
5	Roll back if needed	<code>kubectl rollout undo deployment/<name></code>

DEPLOYMENT STRATEGY OPTIONS

	Recreate	Stop all old pods, then start new ones (simple but causes downtime).
	RollingUpdate	Replace pods gradually (recommended for production).
	maxSurge	Extra pods to create during rollout (e.g. 25%).
	maxUnavailable	Maximum pods that can be unavailable (e.g. 25%).
	Readiness Gates	Wait for custom conditions before marking pod as ready.

SENIOR ENGINEER FOCUS

- Progressive deployment patterns (blue/green, canary).
- Safe rollback and recovery.
- Readiness gates and health checks.
- Surge and unavailable capacity planning.
- PodDisruptionBudget (PDB) interaction.
- Diagnose failed rollouts (events, logs, describe).
- Verify changes in production (pods, metrics, traffic).
- Minimize impact and maintain availability.

COMMON ROLLOUT PROBLEMS

- Rollout stuck or not progressing.
- Pods crash or fail readiness checks.
- New pods not getting traffic.
- Image pull errors.
- Insufficient resources (pending pods).
- PDB blocking rollout.
- Configuration or environment issues.
- Application works in staging but fails in production.

USEFUL CHECKS DURING ROLLOUT

✓	<code>kubectl get deploy</code>	Check deployment status
✓	<code>kubectl get pods</code>	View new pod status
✓	<code>kubectl describe deploy</code>	See events and conditions
✓	<code>kubectl get events</code>	Check rollout related events
✓	<code>kubectl logs <pod></code>	View application logs
✓	<code>kubectl top pods</code>	Check resource usage
✓	<code>kubectl get pdb</code>	Verify PDB settings
✓	<code>kubectl get hpa</code>	Check autoscaling status

REAL-WORLD SCENARIO



"You deploy a new version, but pods keep restarting and users report errors. How do you identify the cause and roll back safely?"

KEY TAKEAWAY

Observe → Change → Watch → Verify → Roll back if necessary.
Make small, safe changes and always have a rollback plan.

PRO TIP

Use `kubectl diff` and staged rollouts to reduce risk. Always monitor metrics, logs and user experience after a deployment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

aws AWS CLI Production Operations

Build a practical cloud investigation command reference.

KEY AWS CLI COMMAND FAMILIES

	<code>aws sts</code>	Identity and account information (who you are)
	<code>aws ec2</code>	Compute instances, networking, security groups
	<code>aws elbv2</code>	Load balancers and target groups
	<code>aws autoscaling</code>	Auto Scaling groups and scaling activities
	<code>aws s3</code>	Object storage and buckets
	<code>aws iam</code>	Users, roles, policies and permissions
	<code>aws cloudwatch</code>	Metrics, alarms and monitoring
	<code>aws logs</code>	Log groups and log streams
	<code>aws route53</code>	DNS records and hosted zones
	<code>aws rds</code>	Databases and clusters
	<code>aws eks</code>	EKS clusters and Kubernetes management

CRITICAL FIRST CHECK



```
$ aws sts get-caller-identity
```

- ✓ Confirms which identity you are using.
- ✓ Shows the AWS account ID.
- ✓ Shows the IAM user or role.
- ✓ Always verify before making any changes.

Never assume you are in the right account or region.

COMMON EXAMPLES

<code>\$ aws ec2 describe-instances</code>	List EC2 instances
<code>\$ aws elbv2 describe-target-health</code>	Check target health
<code>\$ aws s3 ls</code>	List S3 buckets
<code>\$ aws rds describe-db-instances</code>	List RDS instances
<code>\$ aws eks list-clusters</code>	List EKS clusters
<code>\$ aws cloudwatch get-metric-data</code>	Query metrics
<code>\$ aws logs describe-log-groups</code>	List log groups
<code>\$ aws route53 list-hosted-zones</code>	List hosted zones
<code>\$ aws iam list-roles</code>	List IAM roles
<code>\$ aws autoscaling describe-auto-scaling-groups</code>	List ASGs

SENIOR ENGINEER FOCUS

- 1 Identity verification and permissions.
- 2 Region and account mistakes.
- 3 Instance state and health.
- 4 Target health and load balancer issues.
- 5 Logs and metrics for root cause analysis.
- 6 Networking (VPC, subnets, security groups, DNS).
- 7 IAM permissions and least privilege.
- 8 Scaling problems and capacity limits.
- 9 API throttling and retries.
- 10 Service quotas and limits.

COMMON INVESTIGATION AREAS

	Identity	<code>aws sts</code> , <code>aws iam</code>
	Compute	<code>aws ec2</code> , <code>aws autoscaling</code>
	Load Balancing	<code>aws elbv2</code>
	Storage	<code>aws s3</code>
	Database	<code>aws rds</code>
	Kubernetes	<code>aws eks</code>
	Monitoring	<code>aws cloudwatch</code> , <code>aws logs</code>
	Networking	<code>aws route53</code> , VPC, security groups
	Limits	Service quotas, API throttling

REAL-WORLD SCENARIO



"Users cannot access the application. How do you use the AWS CLI to identify the issue, check the target health, review logs and confirm the root cause?"

KEY TAKEAWAY

 Know your identity, check the right region, use the right commands, verify before making changes, and always investigate with logs, metrics and real data.

PRO TIP

 Use `--region`, `--profile` and `--query` to work faster. Always check CloudTrail for critical changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW



Terraform and Infrastructure State Operations

Safely inspect and operate infrastructure as code.

KEY TERRAFORM COMMANDS

\$ terraform init	Initialize working directory and download providers
\$ terraform fmt	Format configuration files
\$ terraform validate	Validate configuration syntax
\$ terraform plan	Show execution plan (what will change)
\$ terraform apply	Apply changes to reach desired state
\$ terraform show	Inspect plan or state file
\$ terraform output	View output values
\$ terraform state list	List resources in state
\$ terraform state show	Show details of a resource in state
\$ terraform state mv	Move resource in state
\$ terraform state rm	Remove resource from state
\$ terraform import	Import existing infrastructure
\$ terraform refresh	Update state from real infrastructure
\$ terraform providers	Show required providers
\$ terraform graph	Visualize resource dependencies

SENIOR ENGINEER FOCUS AREAS

1	State	Understand state file structure, remote state and state backends.
2	Drift	Detect and handle configuration drift.
3	Replacement	Understand when resources will be replaced.
4	Dependencies	Manage resource dependencies and implicit/explicit links.
5	Locking	Use state locking to prevent concurrent changes.
6	Provider Behavior	Understand provider versions and behavior changes.
7	Destructive Plans	Identify and prevent accidental destruction.
8	Sensitive Values	Handle sensitive outputs and secrets safely.
9	Remote State	Use secure remote state (e.g. S3, DynamoDB, Terraform Cloud).

COMMON INVESTIGATION TASKS

Check current state	terraform state list
Verify resource details	terraform state show <resource>
Check what will change	terraform plan
Compare actual infrastructure	terraform refresh
Investigate dependencies	terraform graph
Find provider and version info	terraform providers
Check outputs and values	terraform output
Import existing resources	terraform import <type> <name> <id>

KEY TAKEAWAY



- ✓ Understand the plan before making changes.
- ✓ Know your state and keep it secure.
- ✓ Watch for destructive changes.
- ✓ Use state locking and remote state.
- ✓ Always verify provider versions and resource dependencies.

CRITICAL PRINCIPLE



Never treat terraform apply as the diagnostic step.

Understand the plan first. Always run terraform plan, review the changes, and confirm they are expected before applying.

REAL-WORLD SCENARIO



“A resource is planned for replacement, but you are not sure why. How do you investigate the cause and confirm it is safe to apply the change?”

PRO TIP



- ✓ Use terraform plan -out=plan.tfplan and review with terraform show plan.tfplan.
- ✓ Enable state locking (e.g. S3 + DynamoDB).
- ✓ Use workspaces for environment isolation.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW



Observability and SRE Command Toolkit

Connect infrastructure commands to reliability signals.

KEY COMMANDS AND USAGE

<code>> curl</code>	Test health and metrics endpoints
<code>> Prometheus (queries)</code>	Query metrics (PromQL) via UI or API
<code>> kubectl top</code>	View resource usage in Kubernetes
<code>> docker stats</code>	View container resource usage
<code>> vmstat</code>	System-wide performance overview
<code>> iostat</code>	Disk I/O statistics
<code>> pidstat</code>	Per-process resource usage
<code>> mpstat</code>	CPU usage per core
<code>> ss</code>	Check network connections and ports
<code>> journalctl</code>	View system and service logs
<code>> jq</code>	Parse and filter JSON output

PROMETHEUS QUERY EXAMPLES

<code>rate(http_requests_total[5m])</code>	Request rate
<code>rate(http_requests_total{status=~"5.."}[5m])</code>	Error rate (5xx)
<code>histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))</code>	P95 latency
<code>sum(rate(container_cpu_usage_seconds_total[5m]))</code>	CPU usage
<code>sum(rate(container_memory_usage_bytes[5m]))</code>	Memory usage

REAL-WORLD SCENARIO



"The service is slow and users are complaining. How do you use commands and metrics to find the root cause?"

KEY TAKEAWAY



A command is useful because it tests a hypothesis, not because it appears in a cheat sheet.

THE FOUR GOLDEN SIGNALS

 LATENCY How fast is it?	 TRAFFIC How much is it?	 ERRORS Is it failing?	 SATURATION Is it running out of resources?
• Response time • P95 / P99 latency • End-to-end latency • Upstream latency	• Request rate • Throughput • Concurrent users • Queue depth	• Error rate • HTTP 4xx / 5xx • Retry rate • Timeout rate • Failed requests	• CPU usage • Memory usage • Disk I/O • Network I/O • Queue length • Thread/connection exhaustion

SRE METRICS AND EVIDENCE

 SLO Evidence	Use metrics, logs and traces to prove SLOs.
 Burn Rate	Monitor error budget burn rate.
 Request Rate	Track requests per second (RPS).
 Retry Rate	Monitor retries and their success rate.
 Timeout Rate	Track timeouts and slow requests.
 Queue Depth	Monitor queue length (e.g. Kafka, SQS).
 Resource Saturation	Identify CPU, memory, disk and network limits.
 Cardinality Awareness	Avoid high cardinality in metrics and logs.

SENIOR ENGINEER FOCUS

- 1 Connect infrastructure commands to reliability signals.
- 2 Form and test hypotheses with data.
- 3 Correlate metrics, logs and traces.
- 4 Verify SLOs and error budget impact.
- 5 Identify saturation before it becomes an outage.
- 6 Detect anomalous traffic and failure patterns.
- 7 Understand and control alert noise.
- 8 Use the right tools for the right question.

PRO TIP

- 
 - ✓ Always start with the golden signals.
 - ✓ Use multiple data sources (metrics, logs, traces).
 - ✓ Validate findings with real commands.
 - ✓ Document what you learn.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW

Production Incident Command Playbook

One workflow. All layers. Fast resolution. Long-term prevention.

SCENARIO

 Users report elevated API failures.

THE INCIDENT WORKFLOW

- 1  Confirm symptom
Check alerts, error rates, user reports.
- 2  Define blast radius
Identify affected services, users, regions.
- 3  Check deployments and recent changes
New releases, config changes, rollouts.
- 4  Check latency, traffic, errors, saturation
Use metrics and dashboards.
- 5  Inspect load balancer
Check target health and traffic.
- 6  Inspect Kubernetes workloads
Pods, replicas, events, status.
- 7  Inspect service endpoints
EndpointSlice, readiness, DNS.
- 8  Inspect DNS/network path
DNS, routing, firewalls, network policies.
- 9  Inspect application logs
Get logs, search for errors and traces.
- 10  Inspect node resources
CPU, memory, disk, pressure conditions.
- 11  Check dependencies
DBs, caches, external services.
- 12  Form and test hypothesis
Confirm root cause with evidence.
- 13  Mitigate
Apply fix, scale, rollback, or restart.
- 14  Verify recovery
Confirm service is healthy and stable.
- 15  Monitor
Watch metrics, logs, error rates.
- 16  Capture evidence
Logs, metrics, timeline, commands.
- 17  Prevent recurrence
Root cause fix, improve alerts, update runbooks.

COMMAND DECISION TREE

- `> curl` → Is the service reachable?
- `> dig` → Is DNS correct?
- `> ss` → Are connections healthy?
- `> kubectl get/describe` → Is orchestration healthy?
- `> kubectl logs` → What does the workload report?
- `> top / vmstat / iostat` → Is the host saturated?
- `> journalctl` → What changed at the OS/service layer?
- `> aws elbv2` → Is upstream traffic healthy?
- `> terraform plan` → Is infrastructure drifting?

SENIOR ENGINEER FOCUS

- ✓ Accurate diagnosis, not guesswork.
- ✓ Use evidence, not assumptions.
- ✓ Understand dependencies and blast radius.
- ✓ Balance speed with safety.
- ✓ Communicate clearly with the team.
- ✓ Document the incident and resolution.
- ✓ Turn incidents into lasting improvements.

THE BIG LESSON

 A command is useful because it tests a hypothesis, not because it appears in a cheat sheet.

KEY SIGNALS & METRICS

-  **Latency**
(Response time, p95/p99)
-  **Traffic**
(Request rate, throughput)
-  **Errors**
(Error rate, 4xx/5xx)
-  **Saturation**
(CPU, memory, disk, network)

SRE INDICATORS

-  **SLO evidence**
(availability, latency, success rate)
-  **Burn rate**
(error budget consumption)
-  **Request rate**
(traffic volume)
-  **Retry rate**
(client/server retries)
-  **Timeout rate**
(request timeouts)
-  **Queue depth**
(message/worker queues)
-  **Resource saturation**
(CPU, memory, disk, network)
-  **Cardinality awareness**
(labels, metrics, logs)

PRODUCTION RULE

 Observe → Change → Watch → Verify → Roll back if necessary.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVE | INVESTIGATE | RESOLVE | OPTIMIZE | SCALE

BETTER
ENGINEERS
BUILD A
BRIGHTER
TOMORROW